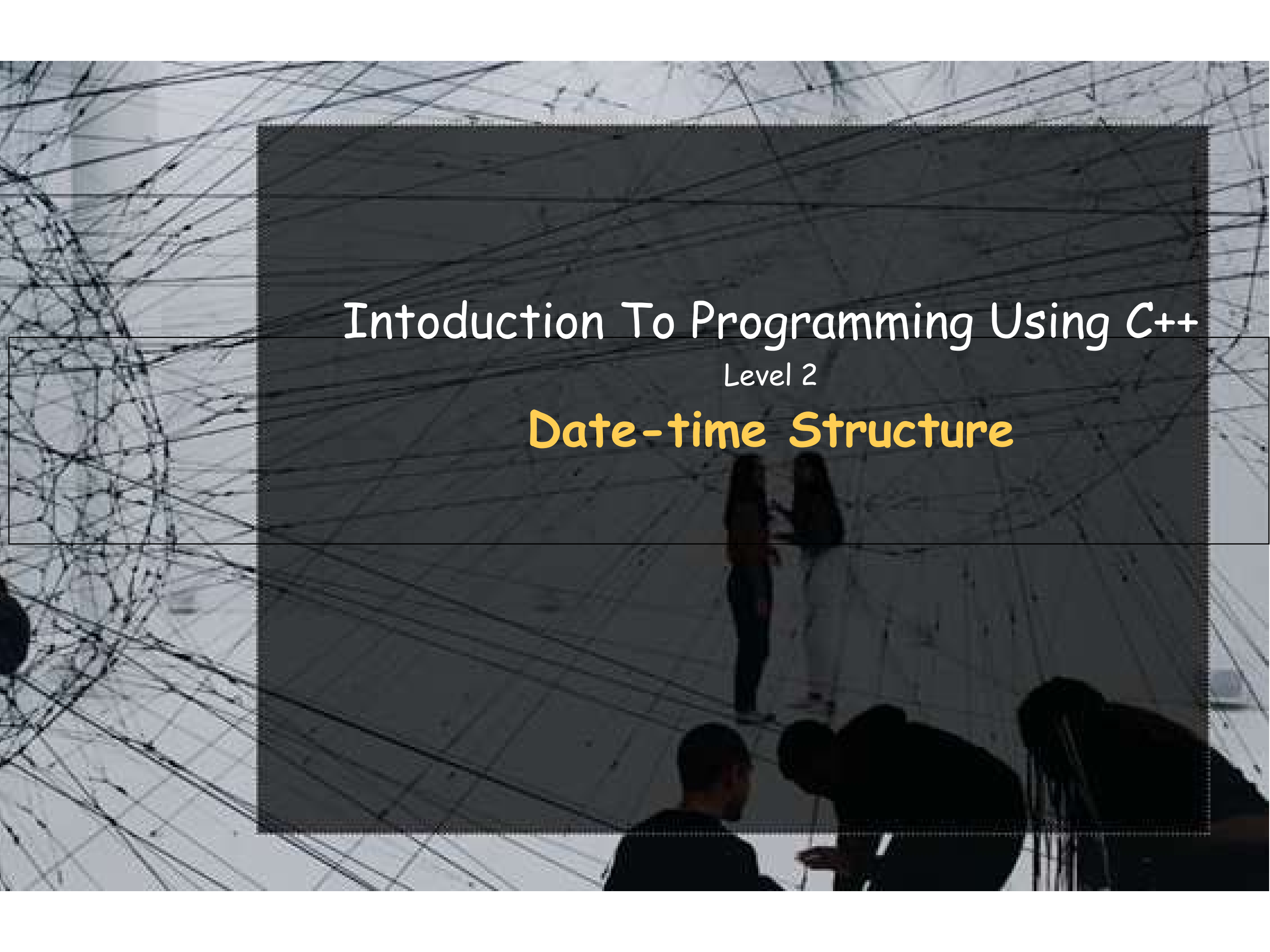




Introduction To Programming Using C++ Level 2

Date-time Structure

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower part of the image, the silhouettes of several people are visible, some standing and some sitting, suggesting a collaborative workspace or a public area. The overall color palette is muted, with greys, blues, and the warm tones of the light coming through the glass.

Introduction To Programming Using C++

Level 2

Date-time Structure

Definition

`tm`

 `tm` is a structure used to store a broken-down representation of date and time.

 It contains separate values for the year, month, day, hour, minute, second, and other date/time information.

Broken-down Time

 Instead of storing time as one large numerical value, the `tm` structure separates it into individual components.

Explanation

Getting the Current Time

 We first use ``time()`` to get the current time as a ``time_t`` value.

```
time_t t = time(0);
```

``localtime()``

 ``localtime()`` converts the ``time_t`` value into a ``tm`` structure representing the local time.

```
tm* nowHere = localtime(&t);
```

``tm_year``

 Represents the number of years since 1900.

 Add 1900 to get the actual year.

``tm_mon``

 Represents the month from 0 to 11.

 Add 1 to get the actual month.

``tm_mday``

 Represents the day of the month directly.

 Values are normally from 1 to 31.

``tm_hour``

 Represents the hour of the day.

 Values are from 0 to 23.

Explanation

`tm\_min`

-  Represents the minutes.

-  Values are from 0 to 59.

`tm\_sec`

-  Represents the seconds.

-  Values are from 0 to 60 depending on implementation and leap-second handling.

`tm\_wday`

-  Represents the day of the week.

-  0 = Sunday, 1 = Monday, ..., 6 = Saturday.

`tm\_yday`

-  Represents the number of days since January 1.

-  Values are from 0 to 365.

`tm\_isdst`

-  Indicates whether daylight saving time is in effect.

-  Positive = daylight saving time is in effect.

-  0 = daylight saving time is not in effect.

-  Negative = information is not available.

Procedures

1. Include the `<ctime>` library.
2. If using Visual Studio and warning C4996 appears, it can be disabled:
`#pragma warning(disable : 4996)`
3. Get the current time:
`time_t t = time(0);`
4. Convert it into local time:
`tm* nowHere = localtime(&t);`
5. Access the individual members of the `tm` structure.
6. Adjust `tm_year` by adding 1900.
7. Adjust `tm_mon` by adding 1.

💡 Example

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    #pragma warning(disable : 4996)
    time_t t = time(0);
    tm* nowHere = localtime(&t);
    cout << "Year: " << nowHere->tm_year + 1900 << endl;
    cout << "Month: " << nowHere->tm_mon + 1 << endl;
    cout << "Day: " << nowHere->tm_mday << endl;
    cout << "Hour: " << nowHere->tm_hour << endl;
    cout << "Minutes: " << nowHere->tm_min << endl;
    cout << "Seconds: " << nowHere->tm_sec << endl;
    cout << "Week Day: " << nowHere->tm_wday << endl;
    cout << "Year Day: " << nowHere->tm_yday << endl;
    cout << "Daylight Saving Time: " << nowHere->tm_isdst << endl;
    return 0;
}
```

💡 Example

➡ Result

Year: 2025

Month: 10

Day: 9

Hour: 14

Minutes: 30

Seconds: 22

Week Day: 4

Year Day: 281

Daylight Saving Time: 1

Note

- ◆ You can use ``gmtime()`` instead of ``localtime()`` if you want the ``tm`` structure to represent UTC instead of local time.
- ◆ The ``tm`` structure stores the components separately, which makes it easier to work with individual parts of a date and time.
- ◆ Some values use zero-based counting, such as ``tm_mon`` and ``tm_wday``.

⊘ Warning

- ◆ Do not forget to add 1900 to ``tm_year`` when displaying the actual year.
- ◆ Do not forget to add 1 to ``tm_mon`` when displaying the actual month.
- ◆ ``tm_wday`` is not the day of the month; it represents the day of the week.

Comparison

``localtime()``

- ✓ Converts ``time_t`` to local date/time.
- ✓ Uses the system's local time zone.
- ✓ Returns a ``tm`` structure.

``gmtime()``

- ✓ Converts ``time_t`` to UTC date/time.
- ✓ Does not use the local time zone.
- ✓ Returns a ``tm`` structure.

✦✦ Summary

- ✓ ``tm`` stores date and time as separate components.
- ✓ ``localtime()`` converts ``time_t`` to local time.
- ✓ ``gmtime()`` converts ``time_t`` to UTC.
- ✓ ``tm_year`` starts from 1900.
- ✓ ``tm_mon`` starts from 0.
- ✓ ``tm_wday`` represents the day of the week.
- ✓ ``tm_yday`` represents the day of the year starting from 0.

Thank you 😊